

Towers of Hanoi: A Complete Tutorial

(For Second Year Undergraduate Computer Science)

1. The Story

In a temple in Hanoi (or, in some versions, at Benares), monks are said to be moving a stack of 64 golden disks between three diamond pegs. The disks were placed on the first peg by the god Brahma at the beginning of time, arranged in decreasing size from bottom to top. The monks work day and night, moving one disk at a time, following two sacred rules:

1. Only one disk may be moved at a time.
2. A larger disk can never be placed on top of a smaller disk.

Legend says that when the monks complete their task — moving all 64 disks to the third peg — the world will end.

This legend was popularized in 1883 by French mathematician **Édouard Lucas**, who invented the puzzle itself and packaged it with this story to make it memorable. It turns out the legend is also a clever way to teach a lesson about exponential growth: even working tirelessly at one move per second, moving 64 disks would take about **585 billion years** — far longer than the current age of the universe. So the monks (and the world) are safe for a while yet.

The puzzle is simple to state, mechanically satisfying to play, and — most importantly for us — a textbook example of how a seemingly hard problem collapses into elegant simplicity once you think about it **recursively**.

2. The Problem, Explained

Setup: Three pegs, labeled A (Source), B (Auxiliary), C (Destination). n disks of different sizes are stacked on peg A, largest at the bottom, smallest at the top.

Goal: Move the entire stack from peg A to peg C.

Rules:

- Move only one disk at a time (the top disk of any stack).
- A disk may only be placed on an empty peg or on top of a larger disk.

- Never place a larger disk on a smaller one.

2.1 Building the Recursive Insight

Suppose you already know how to solve the puzzle for $n - 1$ disks. Can you use that knowledge to solve it for n disks?

Think about the **largest disk** (disk n , at the very bottom of the stack). It cannot move until every disk on top of it is out of the way, and it cannot move again after that. So the largest disk moves **exactly once**, and it must move directly from A to C.

That gives us a three-part plan:

1. **Move the top $n - 1$ disks from A to B**, using C as the auxiliary peg. (This is "get them out of the way.")
2. **Move the remaining largest disk from A to C.** (One single move.)
3. **Move the $n - 1$ disks from B to C**, using A as the auxiliary peg. (Bring them back on top.)

Steps 1 and 3 are themselves instances of the *same problem*, just with $n - 1$ disks and different peg assignments. This is the essence of recursion: we've reduced the problem of size n to two subproblems of size $n - 1$, plus one trivial move.

2.2 Base Case

If $n = 0$, there is nothing to move — do nothing. This stops the recursion.

If $n = 1$, just move the single disk directly from source to destination.

2.3 Worked Example ($n = 3$)

Pegs: A (source), B (auxiliary), C (destination). Disks 1 (smallest), 2, 3 (largest), stacked on A.

Step	Move	A	B	C
0	—	3,2,1	—	—
1	1 → C	3,2	—	1
2	2 → B	3	2	1
3	1 → B	3	2,1	—
4	3 → C	—	2,1	3
5	1 → A	1	2	3
6	2 → C	1	—	3,2
7	1 → C	—	—	3,2,1

Seven moves total for 3 disks — matching the formula we'll derive shortly ($2^3 - 1 = 7$).

3. Pseudocode

3.1 Recursive Solution

```
ALGORITHM Hanoi(n, source, auxiliary, destination)
    // Moves n disks from `source` to `destination`, using `auxiliary` as
    helper

    IF n == 0 THEN
        RETURN // base case: nothing to move
    END IF

    Hanoi(n - 1, source, destination, auxiliary) // step 1: move n-1 disks
    out of the way
    PRINT "Move disk", n, "from", source, "to", destination // step 2: move
    the big disk
    Hanoi(n - 1, auxiliary, source, destination) // step 3: move n-1 disks
    onto destination

// Initial call:
Hanoi(n, 'A', 'B', 'C')
```

Notice how the auxiliary and destination pegs *swap roles* between the two recursive calls — this is the crux of the algorithm and the part students most often get wrong on a first attempt.

3.2 Iterative Solution (for comparison)

An interesting fact: Towers of Hanoi can also be solved iteratively, using a neat bit-pattern trick or a simple stack-based simulation. Here is a stack-based iterative version:

```
ALGORITHM HanoiIterative(n, source, auxiliary, destination)
    totalMoves = 2^n - 1

    IF n is even THEN
        SWAP(auxiliary, destination) // adjust starting parity
    END IF

    FOR i = 1 TO totalMoves DO
        remainder = i MOD 3

        IF remainder == 1 THEN
            MoveDisk(source, destination)
        ELSE IF remainder == 2 THEN
            MoveDisk(source, auxiliary)
        ELSE IF remainder == 0 THEN
            MoveDisk(auxiliary, destination)
        END IF
    END FOR
```

```

        END IF
    END FOR

ALGORITHM MoveDisk(pegX, pegY)
    // Move the top disk between two pegs, respecting the "smaller on larger"
    rule
    IF pegX is empty THEN
        MoveTopDisk(pegY, pegX)
    ELSE IF pegY is empty THEN
        MoveTopDisk(pegX, pegY)
    ELSE IF TopDisk(pegX) > TopDisk(pegY) THEN
        MoveTopDisk(pegY, pegX)
    ELSE
        MoveTopDisk(pegX, pegY)
    END IF

```

The iterative version is useful in practice (no recursion stack overhead, no risk of stack overflow for huge n), but the recursive version is what you should understand deeply — it is the cleanest illustration of "divide and conquer" thinking.

4. Complexity Analysis

4.1 Time Complexity

Let $T(n)$ be the number of moves needed to solve the puzzle for n disks. From the recursive structure:

$$\begin{aligned}
 T(n) &= T(n-1) + 1 + T(n-1) = 2 \cdot T(n-1) + 1 \\
 T(0) &= 0
 \end{aligned}$$

Solving the recurrence (by unrolling or by the standard technique of guessing and verifying):

$$\begin{aligned}
 T(n) &= 2 \cdot T(n-1) + 1 \\
 &= 2(2 \cdot T(n-2) + 1) + 1 = 4 \cdot T(n-2) + 2 + 1 \\
 &= 8 \cdot T(n-3) + 4 + 2 + 1 \\
 &\dots \\
 &= 2^n \cdot T(0) + (2^{n-1} + 2^{n-2} + \dots + 2 + 1) \\
 &= 2^n - 1
 \end{aligned}$$

So:

$$T(n) = 2^n - 1$$

This means the number of moves — and therefore the running time — is:

$O(2^n)$ — exponential time.

This exponential blow-up is *inherent* to the problem, not a flaw in the algorithm: it can be proven that no algorithm can solve the classic 3-peg Towers of Hanoi in fewer than $2^n - 1$ moves, because each of the n disks must move at least once, and the smallest disk alone must move at least 2^{n-1} times in any valid solution. This is why Towers of Hanoi is a favorite example when teaching students the difference between a problem that is *inherently* exponential versus an algorithm that is merely *inefficiently* exponential (like naive recursive Fibonacci, which actually has a polynomial-time solution).

4.2 Space Complexity

Recursive version: The recursion depth is n (each call reduces n by 1 until reaching the base case), and each stack frame uses $O(1)$ space. So the space complexity is:

$O(n)$ — due to the call stack.

Iterative version: Using $O(1)$ extra variables (just counters and peg references) if you don't need to store the peg contents explicitly, giving:

$O(1)$ auxiliary space (excluding the space needed to represent the pegs/disks themselves, which is $O(n)$ regardless of approach).

4.3 Summary Table

Approach Time Space (auxiliary)

Recursive $O(2^n)$ $O(n)$ — call stack

Iterative $O(2^n)$ $O(1)$

Both approaches are exponential in time because the *output itself* (the sequence of moves) has size $2^n - 1$; you cannot solve the puzzle faster than you can *state* the solution.

5. Real-World Applications and Related Ideas

Although you'll probably never move golden disks between diamond pegs professionally, the ideas embedded in Towers of Hanoi show up in surprisingly practical places:

5.1 Teaching Recursion and Divide-and-Conquer

Towers of Hanoi is the canonical first example used to teach recursive thinking, because the recursive structure (solve two smaller identical subproblems + one simple step) maps directly onto more advanced divide-and-conquer algorithms like **Merge Sort**, **Quick Sort**, and **Binary Search**, which students study right after.

5.2 Backup Rotation Schemes

The **Tower of Hanoi backup rotation scheme** is a real algorithm used in data backup systems (particularly for tape backups). It determines which backup media to use on which day so that older backups are overwritten less frequently than newer ones — following the same "small moves often, big moves rarely" pattern that governs the disks (the smallest disk moves every other move; the largest moves only once).

5.3 Graph Theory and the Hanoi Graph

The set of all possible states of an n -disk puzzle, connected by legal moves, forms a well-studied mathematical structure called the **Hanoi graph** (or Sierpiński graph). It's used in graph theory research and has a strikingly fractal, self-similar structure — connecting Towers of Hanoi to fractal geometry.

5.4 Psychology and Cognitive Science

Towers of Hanoi (and its cousin, the **Tower of London** task) is a standard tool in cognitive psychology and neuroscience for measuring **planning ability, working memory, and executive function** — including in clinical assessments for conditions affecting the frontal lobe.

5.5 Genetic Algorithms and AI Planning Benchmarks

Because its optimal solution is well understood and scales predictably, Towers of Hanoi is frequently used as a **benchmark problem** in AI planning research and to test search algorithms (like A*, BFS, or reinforcement learning agents) against a known-optimal number of moves.

5.6 Interleaving Processes / Task Scheduling

The pattern of "pause task A, complete a sub-task fully, resume task A" generalizes to real scheduling problems — such as **disk defragmentation**, robotic arm movement sequencing (moving objects between finite holding areas without violating physical constraints), and certain **compiler register-allocation** problems where values must be shuffled between a limited number of registers/temporary storage locations.

5.7 A Cautionary Tale About Exponential Growth

Beyond direct applications, the puzzle is often invoked pedagogically (like the "wheat and chessboard" legend) to give students intuition for how quickly exponential quantities escape human intuition — relevant to understanding algorithmic complexity, compound interest, viral growth, and Moore's Law-style projections.

6. Key Takeaways

1. **Recursive decomposition:** A problem of size n reduces to two problems of size $n-1$ plus one atomic step — this is the heart of divide-and-conquer.
 2. **Exponential is sometimes unavoidable:** $T(n) = 2^n - 1$ is not a weakness of the algorithm; it is a mathematically provable lower bound.
 3. **Recursive elegance vs. iterative efficiency:** The recursive solution is easiest to *understand*; the iterative solution can be more *space-efficient* in practice.
 4. **Toy problems generalize:** A puzzle invented for entertainment in 1883 now underpins real systems in backup engineering, AI benchmarking, and cognitive science.
-

7. Practice Exercises

1. Prove by induction that $T(n) = 2^n - 1$ satisfies the recurrence $T(n) = 2T(n-1) + 1$ with $T(0) = 0$.
2. Modify the recursive pseudocode to *count* the number of moves without printing each one, and verify it matches $2^n - 1$ for $n = 5$.
3. Trace through the iterative algorithm by hand for $n = 3$ and confirm it produces the same sequence of moves as the recursive version in Section 2.3.
4. Research and explain how the 4-peg variant ("Frame–Stewart algorithm") changes the complexity — and why it remains an area of ongoing mathematical interest.